Deep Learning for Content-Based Filtering 🧠

To implement content-based filtering effectively, we use Neural Networks to transform the raw features of users and items into vectors that can be compared mathematically.

The Architecture: Two Towers

We build two separate neural networks that work together.

1. The User Network

- **Input:** User features vector x_u (Age, Gender, Country, Past behavior, etc.).
- **Hidden Layers:** Dense layers to process the data.
- **Output:** A user vector v_u (also called an **embedding**).
    - *Example Size:* A vector of 32 numbers.

2. The Movie (Item) Network

- **Input:** Movie features vector x_m (Year, Stars, Genre, etc.).
- **Hidden Layers:** Dense layers to process the data.
- **Output:** A movie vector v_m.
    - *Constraint:* This output vector **must** be the same size as the user vector v_u (e.g., 32 numbers).

> ***Note:*** *The two networks can have completely different structures (different numbers of hidden layers or node counts) internally. They only need to match at the* ***final output layer****.*

Making Predictions

Once the networks compute v_u and v_m, we combine them to make a prediction.

A. For Regression (Predicting Star Ratings):
We take the **Dot Product** of the two vectors.
$$\text{Prediction} = v_u \cdot v_m$$

B. For Binary Classification (Predicting Probability of Like/Click):
We apply the **Sigmoid** function to the dot product.
$$\text{Probability} = g(v_u \cdot v_m)$$

Training the Model

We train both networks simultaneously as a single combined model.

The Cost Function (J):
We want to minimize the difference between the predicted rating (dot product) and the actual rating (y).

$$J = \sum_{(i,j):r(i,j)=1} (v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)})^2 + \text{Regularization}$$

- **Process:** We use gradient descent. The error from the cost function backpropagates through the dot product, updating the weights in **both** the User Network and the Movie Network at the same time.
- **Goal:** The networks learn to generate vectors v_u and v_m such that their dot product accurately approximates the user's rating.

Finding Similar Items

A powerful side-effect of this architecture is that the Movie Network learns a vector representation (v_m) for every movie. We can use this to find related items.

- To find movies similar to Movie i:
    1. Take the vector $v_m^{(i)}$.
    2. Search for other movies k where the distance $||v_m^{(k)} - v_m^{(i)}||^2$ is small.
- **Practical Tip:** Since this calculation only depends on movie features (not users), it can be **pre-computed** (e.g., run overnight) and stored. When a user looks at a movie page, the system can instantly look up the pre-calculated "Similar Movies."

Summary

- **Pros:** Very powerful; can combine complex features (text, images, demographics) using neural networks.
- **Cons:** Computationally expensive to run for every user on a massive catalog of items in real-time.